

1. Cover Page

Project Title: Tic Tac Toe

Course: Build Your Own Project

Student Name: RITHIK S

Register Number:26BAS10086

Institution: VIT

Submitted To: Dr.S.DEVARAJU

Date: 30/09/2026

2. Introduction

Tic Tac Toe is one of the simplest two-player strategy games played on a 3×3 grid. The goal is for a player to place three identical symbols (X or O) in a row, a column, or a diagonal. This project turns the traditional paper version into a computer-based version that automatically validates moves, updates the board, and detects wins or draws. It showcases logical thinking, control flow, input handling, and modular programming.

3. Problem Statement

Manual Tic Tac Toe often leads to confusion, like invalid moves, miscounting, or the need to redraw the grid repeatedly. A digital version is required to:

- Eliminate human error
- Automatically detect wins or draws
- Prevent invalid moves
- Provide a clean, user-friendly board

The objective is to create a program that allows two players to play Tic Tac Toe easily and accurately.

4. Functional Requirements

1. Display a 3×3 game board.

2. Allow Player X and Player O to play turn by turn.
3. Validate inputs and prevent illegal moves.
4. Update the board after every turn.
5. Detect winning combinations.
6. Detect draw conditions when the board is full.
7. Display game results.
8. Allow players to restart the game.

5. Non-Functional Requirements

1. Usability: Simple and easy to understand.
2. Reliability: Correct and consistent game results.
3. Performance: Fast input processing with no lag.
4. Maintainability: Code is modular and easy to modify.
5. Error Handling: Handles wrong inputs without crashing.

6. System Architecture

The Tic Tac Toe system consists of the following modules:

Input Module: Gets user positions (1–9).

Validation Module: Checks if the move is allowed.

Game Logic Module: Detects wins or draws.

Board Module: Updates and displays the grid.

Controller: Manages turns and game flow.

This architecture ensures smooth and structured gameplay.

7. Design Diagrams

7.1 Use Case Diagram

Actors: Player X, Player O

Use Cases:

- Start Game
- Make Move
- Validate Move
- Update Board
- Check Win/Draw
- Restart Game

7.2 Workflow Diagram

1. Start game
2. Show empty board
3. Player enters a move
4. Validate move
5. Update board
6. Check win or draw
7. Switch player
8. Repeat until the game ends
9. Show result

7.3 Sequence Diagram

Player → Game System → Validator → Board → Result

1. Player inputs position
2. Validator checks input
3. Board updates
4. Game checks win or draw

5. Display result

6. Switch turn

7.4 Class / Component Diagram

Classes/Modules:

GameManager

- startGame()
- switchPlayer()
- checkResult()

Board

- displayBoard()
- updateBoard()

Validator

- isValidMove()

7.5 ER Diagram (if storage used)

Tic Tac Toe usually does not use storage, so an ER Diagram is not required.

(If needed, you can show a simple table for storing match history.)

8. Design Decisions & Rationale

Console-based interface: Simple to build and understand.

3×3 array/list: Easy for indexing and updating positions.

Modular design: Makes the game easy to maintain and extend.

Immediate win checking: Improves game speed and reduces unnecessary loops.

Player switching logic: Ensures fairness and correctness.

9. Implementation Details

Implemented using Python / Java / C / C++ (choose your language).

Main data structure: 3×3 list/array.

Key functions include:

- displayBoard()
- checkWin()
- checkDraw()
- isValidMove()
- updateBoard()
- switchPlayer()

The game runs in a loop until there is a win or a draw. The program handles invalid numeric and non-numeric inputs.

10. Screenshots / Results

Include screenshots of:

- Starting game board
- Player X move
- Player O move
- Winning scenario
- Draw scenario

(You can add these later when you run your program.)

11. Testing Approach

Unit tests:

- Win detection
- Draw detection
- Move validation

Manual testing:

- All 8 winning patterns tested
- Invalid moves tested
- Boundary inputs tested
- Repeated moves tested

12. Challenges Faced

- Handling invalid inputs safely
- Ensuring clean and readable board output
- Implementing all win conditions correctly
- Ensuring the game flow switches properly between players
- Structuring the program in modular functions

13. Learnings & Key Takeaways

- Improved understanding of programming logic
- Better handling of user input
- Learned modular programming concepts
- Gained experience in debugging
- Understood how game logic is implemented

14. Future Enhancements

- Add AI opponent using Minimax Algorithm
- Create GUI version using Tkinter / JavaFX / HTML+CSS+JS
- Add scoreboard to track wins
- Add sound effects and animations
- Add multiplayer mode over the internet

15. References

Class notes and lecture materials

W3Schools / GeeksForGeeks (basic logic concepts)

Python/Java/C documentation

Online Tic Tac Toe algorithm resources

